编译原理与技术课程设计需求分析报告

1. 项目概述

这次课程设计要完成的是一个面向 Pascal-S 语言的编译程序。整体上需要把源程序输入、词法分析、语法分析、语义分析、代码生成和测试验证这一整套流程做完整，最后把 Pascal-S 源程序翻译成能够编译运行的 C 语言程序。根据课程要求，这个项目不只是做一个能识别语法的程序，还要做出程序实现、测试、设计文档和验收材料。

2. 功能性需求

2.1 输入与基本处理

系统要能接收 Pascal-S 源程序，并按照课程给定的语法和语义规则进行处理。输入主要有两种来源：

1. 本地命令行输入源文件路径。
2. 平台自动评测中传入的测试用例文件。

系统还需要把源文件翻译成同名的 C 文件，并保存到源文件所在目录。

2.2 词法分析需求

词法分析模块需要完成下面这些事：

1. 识别 Pascal-S 语言中的关键字、标识符、常量、运算符、界符等单词符号。
2. 处理注释和分隔符。
3. 为各类单词定义内部编码及属性值。
4. 记录单词在源程序中的位置信息，至少包含行号和列号，以便错误定位。
5. 在发现词法错误时能够输出明确的错误提示。
6. 词法分析器应作为语法分析器的输入接口，返回词汇类别编码与属性值。

2.3 语法分析需求

语法分析模块需要满足以下要求：

1. 按照 Pascal-S 语言文法对源程序进行语法分析。
2. 支持对语法错误进行检测，并给出必要的错误提示。

3. 分析方法可以采用自顶向下预测分析、自底向上 LR 分析，或者使用 YACC 工具实现。
4. 若采用手工实现，需要完成文法处理、分析表构造或规则设计等工作。
5. 语法分析结果应为后续语义分析和代码生成提供结构化中间结果。

2.4 语义分析需求

语义分析和符号表这部分需要做到：

1. 构建并维护符号表。
2. 支持符号查找、插入、定位和重定位等操作。
3. 完成变量、函数、过程、数组、表达式等语义检查。
4. 检查类型匹配、作用域、参数传递以及调用关系等语义问题。
5. 对语义错误给出可定位、可理解的报错信息。

2.5 代码生成需求

代码生成模块需要把 Pascal-S 翻译成 C 语言：

1. 输出与输入源文件同名的 C 文件。
2. 翻译结果应可被后续 C 编译器编译运行。
3. 对源语言与目标语言之间的映射关系进行合理设计。
4. Pascal 的 Real 类型需要翻译为 C 的 float，而不是 double。
5. 在翻译浮点数相关行为时，应以功能正确为准，不要求 Pascal 与 C 的运行细节完全一致。

2.6 错误检测与恢复需求

词法、语法和语义阶段都要做错误检测，并尽量支持错误恢复：

1. 能识别常见词法错误、语法错误和语义错误。
2. 错误提示应尽量准确、具体，并包含定位信息。
3. 对部分错误应具备继续分析的能力，以便报告更多错误而不是在首个错误处直接终止。

2.7 测试与验证需求

系统还需要能支持完整的测试流程：

1. 支持单元测试与集成测试。
2. 支持对功能模块分别构造测试用例。
3. 支持对错误用例进行验证，检查错误检测与恢复效果。
4. 支持对生成的 C 程序进行编译和运行验证。
5. 测试结果需要记录并分析。

3. 非功能性需求

3.1 正确性

系统必须保证编译语义正确，能够将符合 Pascal-S 语法与语义约束的源程序稳定翻译为语义等价的 C 程序。词法、语法、语义和代码生成各阶段的处理结果应保持一致性，不得出现前后阶段定义冲突。对课程平台提供的测试数据，系统应达到可复现、可通过的结果，并保证生成代码可编译、可运行、输出行为与设计预期一致。

3.2 可维护性

系统采用模块化架构实现，至少划分为词法分析、语法分析、语义分析、代码生成、错误处理与测试支持等独立模块。模块之间通过清晰接口交互，输入输出数据结构保持稳定，避免跨模块的隐式依赖。各模块代码应具备明确的职责边界和必要注释，便于后续定位问题、替换实现和增量优化。

3.3 可测试性

系统必须支持分层测试与结果复现。测试范围覆盖正常路径、语法错误路径、语义错误路径、边界输入以及复杂嵌套结构。对每个测试用例，应能够明确记录输入程序、预期输出/预期报错、实际结果和差异分析。编译流程应支持批量执行与回归验证，以便在修改局部模块后快速确认全局功能未回退。

3.4 可移植性与平台适配

系统必须严格适配课程平台运行环境与评测流程，确保在指定工具链下可直接构建与执行。平台环境以指导手册为准，包括 Debian GNU/Linux 9、Flex 2.6.1、Bison 3.0.4、cmake 3.28.2、Python 3.7.5 和 c++ 7.3.0。与平台运行相关的可执行文件命名、命令行参数、输出文件位置等行为应保持稳定，避免因环境差异导致评测失败。

3.5 工程规范

项目实现与文档编写均应遵循统一工程规范。代码层面要求命名语义清晰、缩进风格一致、关键逻辑有必要注释、目录结构与模块划分可追踪；文档层面要求需求、设计、实现、测试之间术语一致、内容可对应、结论可验证。所有提交物应满足可阅读、可复查、可复现实验结果的基本质量要求。

4. 开发与实现约束

4.1 推荐开发方式

整个课程设计按照软件工程的思路分阶段推进：

1. 先进行需求分析和总体设计。
2. 再进行详细设计与模块实现。
3. 之后进行单元测试、集成测试和系统测试。
4. 最后整理文档并完成验收答辩。

4.2 推荐工具与方法

语法分析采用 YACC / Bison 实现，开发时配合 Flex、Bison 和 cmake 完成整体构建。

4.3 平台评测约束

指导手册对自动评测有比较明确的要求：

1. 可执行编译器文件必须命名为 pascc。
2. 可执行文件需要放置在 /data/workspace/myshixun/bin 目录下。
3. 编译器应支持命令行参数 -i filename.pas。
4. 运行后应生成同名的 .c 文件，并保存到源文件所在目录。
5. 评测时会同时使用 open_set 和 hidden_set 两类测试集。

5. 交付物需求

这次课程设计最后需要提交的内容包括：

1. 需求分析报告。
2. 总体设计报告，包括软件功能描述、模块划分、软件结构图、符号表结构设计和模块接口定义等。
3. 详细设计报告，包括模块功能、输入输出和处理逻辑等。
4. 源程序与可执行程序。
5. 测试报告，包括测试计划、测试用例、测试结果和分析。
6. 课程设计答辩 PPT。
7. 课程设计报告完整版。
8. 程序及使用说明，包括源程序、可运行程序、测试用例程序和使用说明。
9. 填写完整并打印的《验收登记表》。

6. 项目组织需求

6.1 团队规模与分工

这次课程设计由四人（金介然、丁乐航、何俊辉、李璇）组成，并明确一名组长（丁乐航）负责整体协调。

6.2 分工建议

四个人的分工如下：

1. 第一位成员负责词法分析模块，以及测试用例的编写和整理，具体包括单词符号识别、Token 定义、错误定位和词法测试。
2. 第二位成员负责语法分析与语义分析的手工编码实现，具体包括文法整理、分析过程实现、语法错误提示、符号表维护、作用域管理、类型检查和参数检查。
3. 第三位成员负责语法制导翻译方案的实现，采用 YACC / Bison 完成语法制导翻译相关工作，并把翻译过程和语法分析结果衔接起来。
4. 第四位成员负责代码生成、整体联调和测试验证，具体包括 Pascal-S 到 C 的翻译规则、生成文件检查、样例测试、回归测试和最终集成；该成员兼任组长，统筹软件集成测试和整体进度。

6.3 成员要求

每个成员都要有明确而且相对独立的任务，并完成对应模块的详细设计、编码和单元测试。组长还要负责把系统集成起来，组织联调和集成测试，保证最后交上去的程序和文档是对得上的。